

Part A

1.

```
#include <iostream>

using namespace std;

class BankAccount {
public:
    BankAccount() {
        cout << "Bank account created." << endl;
    }
    ~BankAccount() {
        cout << "Bank account closed." << endl;
    }
};

int main() {
    BankAccount acc; // Object created
    // Goes out of scope at end of main()
    return 0;
}
```

Output:

Bank account created.

Bank account closed.

2.

```
#include <iostream>

using namespace std;

class Book {
    string title;
    float price;
public:
    // Constructor with parameters
```

```

Book(string t, float p) {
    title = t;
    price = p;
}

void display() {
    cout << title << endl;
    cout << price << endl;
}

};

int main() {
    Book b("C++ Programming", 499.99);
    b.display(); // Call display function to show book details
    return 0;
}

```

Output:

C++ Programming

499.99

3.

```

#include <iostream>

using namespace std;

// Enum to represent traffic signal states
enum Signal { RED, YELLOW, GREEN };

// Function to simulate the traffic signal
void showSignal(Signal signal) {
    switch(signal) {
        case RED:
            cout << "Stop" << endl;
            break;
        case YELLOW:

```

```

        cout << "Ready" << endl;

        break;

    case GREEN:

        cout << "Go" << endl;

        break;

    default:

        cout << "Invalid Signal" << endl;

    }

}

int main() {

    Signal currentSignal;

    // Simulate Red Signal
    currentSignal = RED;
    showSignal(currentSignal);

    // Simulate Yellow Signal
    currentSignal = YELLOW;
    showSignal(currentSignal);

    // Simulate Green Signal
    currentSignal = GREEN;
    showSignal(currentSignal);

    return 0;

}

```

Output:

Stop

Ready

Go

4.

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;

class Book {
    string title;
public:
    Book(string t) : title(t) {}
    void display() const {
        cout << "Book Title: " << title << endl;
    }
};

class Library {
    vector<Book*> books; // Aggregation: Library has pointers to Book
public:
    void addBook(Book* book) {
        books.push_back(book);
    }
    void showBooks() const {
        cout << "Books in the Library:" << endl;
        for (auto book : books) {
            book->display();
        }
    }
};

int main() {
    // Creating independent Book objects
    Book b1("C++ Programming");
    Book b2("Data Structures");
    // Creating Library and adding books
    Library lib;
    lib.addBook(&b1);
```

```

lib.addBook(&b2);

lib.showBooks();

return 0;

}

```

Output:

Books in the Library:

Book Title: C++ Programming

Book Title: Data Structures

5.

 Purpose of VFT in C++:

Enable Dynamic Dispatch:

VFT allows the program to decide at runtime which version of a virtual function to call, based on the actual object type, not the pointer or reference type.

Support Virtual Functions:

When a class has virtual functions, the compiler creates a table (VTable) containing pointers to the actual functions.

Each object of the class gets a hidden pointer to this VTable, called the vptr.

Facilitate Inheritance and Overriding:

If a derived class overrides a base class's virtual function, its VTable will have a pointer to the overridden function instead of the base one.

Code:

```

class Base {
public:
    virtual void show() { cout << "Base show" << endl; }
};

class Derived : public Base {
public:
    void show() override { cout << "Derived show" << endl; }
};

```

6.

 Static vs Dynamic Binding in C++

Feature	Static Binding (Early Binding)	Dynamic Binding (Late Binding)
Resolution Time	Compile-time	Runtime
Keyword	No special keyword	Requires <code>virtual</code> keyword
Function Type	Non-virtual functions	Virtual functions
Supports Polymorphism	 No	 Yes
Performance	Faster (no runtime overhead)	Slightly slower (due to VTable lookup)
Flexibility	Less flexible	More flexible (supports dynamic behavior)
Example	Function overloading, normal calls	Overriding virtual functions
Binding Mechanism	Direct function call	VTable and vptr-based lookup
Usage Scenario	When behavior is known at compile-time	When behavior depends on actual object type

Code:

```
class A {
public:
    void display() { cout << "A's display\n"; }
};

int main() {
    A obj;
    obj.display(); // Static binding
}

class A {
public:
    virtual void display() { cout << "A's display\n"; }
};

class B : public A {
public:
    void display() override { cout << "B's display\n"; }
```

```
};

int main() {
    A* obj = new B();
    obj->display(); // Dynamic binding
}
```

7.

```
#include <iostream>
#include <string>
using namespace std;
// Department class
class Department {
    string name;
public:
    Department(string deptName) : name(deptName) {}
    string getName() const {
        return name;
    }
    void display() const {
        cout << "Department: " << name << endl;
    }
};

// Student class
class Student {
    string name;
    int rollNo;
    Department& dept; // Aggregation: reference to Department
public:
    Student(string studentName, int r, Department& d)
        : name(studentName), rollNo(r), dept(d) {}
};
```

```

void display() const {
    cout << "Student Name: " << name << endl;
    cout << "Roll Number: " << rollNo << endl;
    dept.display();
}
};

int main() {
    // Create a department independently
    Department cs("Computer Science");
    // Create students and associate them with the department
    Student s1("Alice", 101, cs);
    Student s2("Bob", 102, cs);
    // Display student info
    s1.display();
    cout << "-----" << endl;
    s2.display();
    return 0;
}

```

Output:

Student Name: Alice

Roll Number: 101

Department: Computer Science

Student Name: Bob

Roll Number: 102

Department: Computer Science

8.

```
#include <iostream>
```

```
using namespace std;
```



```

class Calculator {
public:
    // Function to add two integers
    int add(int a, int b) {
        return a + b;
    }
    // Function to add two floating-point numbers
    float add(float a, float b) {
        return a + b;
    }
    // Optional: Function to add one integer and one float
    float add(int a, float b) {
        return a + b;
    }
    float add(float a, int b) {
        return a + b;
    }
};

int main() {
    Calculator calc;

    // Integer addition
    int intResult = calc.add(5, 10);
    cout << "Integer Addition: 5 + 10 = " << intResult << endl;

    // Float addition
    float floatResult = calc.add(3.5f, 2.5f);
    cout << "Float Addition: 3.5 + 2.5 = " << floatResult << endl;

    // Mixed addition
    float mixedResult1 = calc.add(4, 2.5f);

```

```

    cout << "Mixed Addition (int + float): 4 + 2.5 = " << mixedResult1 << endl;
    float mixedResult2 = calc.add(3.5f, 2);
    cout << "Mixed Addition (float + int): 3.5 + 2 = " << mixedResult2 << endl;
    return 0;
}

```

Output:

Integer Addition: $5 + 10 = 15$

Float Addition: $3.5 + 2.5 = 6$

Mixed Addition (int + float): $4 + 2.5 = 6.5$

Mixed Addition (float + int): $3.5 + 2 = 5.5$

9.

```

#include <iostream>
#include <iomanip> // for setprecision
using namespace std;

int main() {
    int totalCost, itemCount;

    // Input
    cout << "Enter total cost: ";
    cin >> totalCost;
    cout << "Enter item count: ";
    cin >> itemCount;

    // Calculate average price (cast to float for correct division)
    float averagePrice = static_cast<float>(totalCost) / itemCount;

    // Output with 2 decimal places
    cout << fixed << setprecision(2);
    cout << "Average Price: " << averagePrice << endl;
    return 0;
}

```

Input:

Enter total cost: 250

Enter item count: 4

Output:

Average Price: 62.50

Part B

1.

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Student {
```

```
private:
```

```
    string name;
```

```
    int id;
```

```
    int age;
```

```
    string* courses; // dynamically allocated array of courses
```

```
    int courseCount;
```

```
public:
```

```
    // Constructor
```

```
    Student(string n, int i, int a, string courseList[], int count) {
```

```
        name = n;
```

```
        id = i;
```

```
        age = a;
```

```
        courseCount = count;
```

```
        courses = new string[courseCount];
```

```
        for (int j = 0; j < courseCount; j++) {
```

```
            courses[j] = courseList[j];
```

```
        }
```

```
}
```

```
// Copy Constructor (Deep Copy)
```

```
Student(const Student& other) {  
    name = other.name;  
    id = other.id;  
    age = other.age;  
    courseCount = other.courseCount;  
    courses = new string[courseCount];  
    for (int i = 0; i < courseCount; i++) {  
        courses[i] = other.courses[i];  
    }  
}
```

```
// Assignment Operator Overloading (Deep Copy)
```

```
Student& operator=(const Student& other) {  
    if (this != &other) {  
        // Free existing memory  
        delete[] courses;  
  
        name = other.name;  
        id = other.id;  
        age = other.age;  
        courseCount = other.courseCount;  
        courses = new string[courseCount];  
        for (int i = 0; i < courseCount; i++) {  
            courses[i] = other.courses[i];  
        }  
    }  
}
```

```

        return *this;
    }

// Display student info
void display() const {
    cout << "Name: " << name << endl;
    cout << "ID: " << id << endl;
    cout << "Age: " << age << endl;
    cout << "Courses Enrolled:" << endl;
    for (int i = 0; i < courseCount; i++) {
        cout << " - " << courses[i] << endl;
    }
}

// Destructor
~Student() {
    delete[] courses;
}

};

int main() {
    string courses1[] = {"Math", "Physics", "Chemistry"};
    Student s1("Alice", 101, 20, courses1, 3);

    cout << "Original Student Record (s1):" << endl;
    s1.display();

    // Creating a new student using the copy constructor
    Student s2 = s1;

```

```

cout << "\nCopied Student Record (s2 - via Copy Constructor):" << endl;
s2.display();

// Modify s2 to prove independence
string courses2[] = {"Biology", "English"};
Student s3("Bob", 102, 21, courses2, 2);

// Assign s3 to s2 using assignment operator
s2 = s3;
cout << "\nUpdated s2 using Assignment Operator with s3's data:" << endl;
s2.display();

cout << "\nOriginal s1 remains unchanged:" << endl;
s1.display();

return 0;
}

```

Output:

Original Student Record (s1):

Name: Alice

ID: 101

Age: 20

Courses Enrolled:

- Math
- Physics
- Chemistry

Copied Student Record (s2 - via Copy Constructor):

Name: Alice

ID: 101

Age: 20

Courses Enrolled:

- Math
- Physics
- Chemistry

Updated s2 using Assignment Operator with s3's data:

Name: Bob

ID: 102

Age: 21

Courses Enrolled:

- Biology
- English

Original s1 remains unchanged:

Name: Alice

ID: 101

Age: 20

Courses Enrolled:

- Math
- Physics
- Chemistry

2.i

Code:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Base class Vehicle

class Vehicle {
protected:
    string brand; // Property to store the brand of the vehicle

public:
    // Setter function to set the brand
    void setBrand(string b) {
        brand = b;
    }

    // Getter function to get the brand
    string getBrand() const {
        return brand;
    }
};
```

```
// Derived class Car from Vehicle

class Car : public Vehicle {
private:
    string model; // Property to store the model of the car

public:
    // Setter function to set the model
    void setModel(string m) {
        model = m;
    }

    // Getter function to get the model
```



```
string getModel() const {  
    return model;  
}  
  
// Method to display brand and model  
void display() const {  
    cout << getBrand() << endl;  
    cout << getModel() << endl;  
}  
};  
  
int main() {  
    string brand, model;  
  
    // Input  
    cout << "Enter brand: ";  
    getline(cin, brand);  
    cout << "Enter model: ";  
    getline(cin, model);  
  
    // Create an instance of Car  
    Car myCar;  
    myCar.setBrand(brand);  
    myCar.setModel(model);  
  
    // Display the brand and model of the car  
    myCar.display();  
  
    return 0;
```

```
}
```

Input:

Enter brand: Toyota

Enter model: Camry

Output:

Toyota

Camry

ii.

code:

```
#include <iostream>
```

```
using namespace std;
```

```
// Complex number class
```

```
class Complex {
```

```
private:
```

```
    int real, imag; // real and imaginary parts of the complex number
```

```
public:
```

```
    // Constructor to initialize complex number
```

```
    Complex(int r, int i) : real(r), imag(i) {}
```

```
    // Overload + operator to add two complex numbers
```

```
    Complex operator+(const Complex& other) {
```

```
        // Adding real parts and imaginary parts separately
```

```
        return Complex(real + other.real, imag + other.imag);
```

```
    }
```

```
    // Function to display complex number in the format: a + bi
```

```
    void display() const {
```

```

        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    int r1, i1, r2, i2;

    // Input for first complex number
    cin >> r1 >> i1;
    // Input for second complex number
    cin >> r2 >> i2;

    // Create complex number objects
    Complex c1(r1, i1);
    Complex c2(r2, i2);

    // Add two complex numbers using overloaded + operator
    Complex result = c1 + c2;

    // Output the result
    result.display();

    return 0;
}

```

Input:

3 4

1 2

Output:

4 + 6i

3.

Code:

```
#include <iostream>

#include <string>

using namespace std;

// Abstract base class Employee
class Employee {
protected:
    string name; // Employee name

public:
    // Constructor to initialize the name
    Employee(string n) : name(n) {}

    // Pure virtual function to calculate salary
    virtual int calculateSalary() = 0;

    // Function to display employee name and salary
    void display() {
        cout << name << endl;
        cout << calculateSalary() << endl;
    }

    // Virtual destructor for proper cleanup
    virtual ~Employee() {}
};

// Derived class FullTimeEmployee that inherits from Employee
```

```

class FullTimeEmployee : public Employee {
private:
    int monthlySalary; // Monthly salary of the full-time employee

public:
    // Constructor to initialize name and monthly salary
    FullTimeEmployee(string n, int salary) : Employee(n), monthlySalary(salary) {}

    // Implement calculateSalary method
    int calculateSalary() override {
        return monthlySalary;
    }
};

int main() {
    string name;
    int monthlySalary;

    // Input employee details
    cin >> name;
    cin >> monthlySalary;

    // Create a FullTimeEmployee object
    FullTimeEmployee employee(name, monthlySalary);

    // Display employee details
    employee.display();

    return 0;
}

```

}

Input:

Alice

50000

Output:

Alice

50000